

CS6650 Assignment 4

Distributed Databases using Replication

Remember to turn off all your instances and resources etc in AWS when you finish each day.

This is a **team assignment**. You can use the same team as assignment 3, or change teams. Create teams of four or five and inform the TA about your team. Teams should be created before the next class on April-2. Any students not in a team will be organized into teams by us – you don't want that so start early! Because this is a team assignment, you only need to have one set of instances running in AWS. Only one member of each team needs to submit to Canvas

This is a difficult assignment, so start early!

Distributed Databases using Replication

This assignment cuts to heart of distributed systems – how to work with the CAP theorem to safely manage shared state in a distributed system.

Key-Value (KV) Service

You will write an in-memory Key-Value store as the foundation for a Leader-Follower distributed database and a Leaderless distributed database. Both these databases are essentially hashtable-as-a-service; a simplified version of Redis. The API for both databases is identical.

The api has a single endpoint `/kv` that accepts PUT (for writes) and GET (for reads).

The API has two very simple endpoints:

1. PUT store the `value` under that key, and returns `201-Created`. The key cannot be empty. The empty string is a valid value. This endpoint is also known as *write*.
2. GET returns the value (and `200-OK`) if the Key is present, otherwise return 404. This is also known as *read*.

For example:

```
POST /kv
Content-Type: application/json

Request Body:
{ "key": "mykey", "value": "myvalue" }

Response: 201 Created
{ "key": "mykey", "version": 1 }
```

For simplicity the KV service does NOT need to persist any data to the file system; all values are stored in memory and are lost when the service shuts down.

You will need to create Dockerfiles etc and deploy your services just like you did for the previous assignments.

The PUT and GET actions of the KV service **must** add delays to simulate the real-world time delays that would occur when writing to storage, and make it easier to test the inconsistency window. The reason for adding the delays is that if everything commits really quickly then the race condition windows are very small, and it is hard to create conflicts. The goal of this assignment is to show how databases gradually fail under load because of the delays in propagating the writes.

Specifically, when the Leader is sequentially updating its Followers during a PUT operation, the Leader must sleep for 200ms to simulate its own write to permanent storage **after it has sequentially sent a PUT messages to each of its Followers. Each Follower must sleep for 200ms when it receives the PUT update before responding.** Total write time for $W=5$ with 4 followers: approximately $5 \times 200\text{ms} = 1.0 \text{ second}$.

When any node (Leader or Follower) receives a GET operation it must sleep for 50ms before responding. In addition, if $R > 1$ the Leader must sequentially send GET requests to the required number of Follower nodes.

Leader-Follower Database

You **must** add a configuration file or command line parameters so that your database can be easily configured for different values of N , R , and W as defined in the lecture. You **must** have five nodes in the set of nodes that make up the distributed database, i.e. $N == 5$. That is one Leader and four Followers.

For example: Use a configuration file or environment variables to configure each node:

Leader-Follower:

```
ROLE: "leader" or "follower"
FOLLOWER_URLS: Comma-separated list of follower URLs (for leader only)
WRITE_QUORUM_SIZE: Write quorum (1, 3, or 5)
READ_QUORUM_SIZE: Read quorum (1, 3, or 5)
```

Leaderless:

`PEER_URLS`: Comma-separated list of all other node URLs

All write requests go to the Leader, which replicates the data to all other nodes. Therefore, your load-test client needs to know the IP address of the Leader and only send writes to it. Reads can go to any instance.

Replication Strategies

You **must** implement the following replication strategies:

1. $W=5$ and $R=1$, meaning that on a write every node **must** be updated before the Leader responds to the client. A read only needs to read the value on the leader node before responding to the client.
2. $W=1$ and $R=5$, meaning that only the Leader needs to be updated before responding to a write request. However, a read **must** fetch the data from every node and return the most recent version.
3. Use a quorum of $R=3$, $W=3$. Writes **must** update at least three nodes before responding to the client. Reads **must** return the most recent version of the data after getting the value from three Follower nodes.

You will need logical version numbers on the KV pairs for these algorithms, see lecture notes for Module 9. That logical version number is an integer which follows these rules:

- Each key starts at version 1 when first written.
- Each subsequent write to that key increments its version number.
- In Leader-Follower: Only the Leader assigns version numbers.
- In Leaderless: The Write Coordinator assigns the new version number before propagating.
- When $R > 1$, the node coordinating the read **must** collect responses from R nodes and return the value with the highest version number.

Leaderless Database

This is the same situation except that there is no distinguished leader. Read and Write requests can go to any node. You **must** use a load balancer or some other mechanism to ensure that read and write requests from the client are spread evenly across all the nodes. Configure the case where $W=N$ and $R=1$. The node performing the read **must** sleep for 50 ms when it receives a read request.

When a node receives a write request it becomes the **Write Coordinator** for that write request. It **must** coordinate writes to every other node, wait for them to complete, and only then return `201-Created` to the client.

Propagation delays are the same – every node sleeps 200ms when receiving a write, every node sleeps 50ms when receiving a read.

When a node receives a read request from the client it returns its own value. All writes must be propagated by the node that is the Write Coordinator to all other nodes. It is

possible for a read to arrive at an un-updated node, and therefore there is an inconsistency window. This flaw is acceptable because the purpose of the following section is to show this inconsistency happening in practice.

Testing Consistency

You **must** write three unit tests that test the Leader database for consistency:

Test 1: Strong consistency with W=5

A basic smoke test.

1. Send PUT to the Leader Node.
2. After it acknowledges, a read to the same Leader node should return that new value.

Test 2: Strong consistency with W=5

1. Configure W=5, R=1.
2. Send PUT to the Leader and wait for acknowledgment.
3. Read from any Follower—should return the new value.

Test 3: Exposing inconsistency with W=1

1. Configure W=1, R=1.
2. Send PUT to the Leader.
3. Immediately (i.e. within $5 \times 200\text{ms} = 1\text{s}$) send `local_read` to Followers.
4. Some Followers should return stale data (or 404 for the first write to that key).

To sneakily test the internal workings of the Leader database, add an endpoint `local_read` which returns the KV value on that node only. During a set operation that endpoint will often return inconsistent data. Adding sneaky (API-breaking) read methods like `local_read` is a common testing tactic, often on a subclass that is only used during testing.

1. Send PUT to the Leader.
2. Within the update window the test sends `local_read` to the other Followers. This might return inconsistent data. If you repeat this often at high load it should happen.

For the Leaderless database, write a unit test that exposes the inconsistency window:

1. The unit test writes a key-value pair to a random node. That node is now the Write Coordinator.
2. Within the update time window send read requests (i.e. send `get` messages) to other nodes. Some of these values returned should be inconsistent.
3. After the Coordinator acknowledges the write, read from the Coordinator. That should be consistent.
4. After the Coordinator acknowledges the write, read from another node. That should be consistent.

Load Testing

You **must** modify your load test client from Assignment 2 to explore the read-write performance of the various databases. In order to produce stale reads, you **must** ensure that your load test generator produces data that is "local-in-time", i.e. **reads and writes to the same key are clustered closely together in time**. Some suggestions: you can either simply use a smaller number of keys (easy) or a more complex algorithm that clusters reads and writes to the same key closely together in time, but still has a large number of keys.

You must perform load-test runs with the following read-write ratios on the three Leader and one Leaderless configuration and record your results:

Writes	Reads
1%	99%
10%	90%
50%	50%
90%	10%

For each run you must record the:

1. Latency for each request, and
2. Number of stale reads from Followers or non-Write Coordinator nodes.

You will need to track the version of the data in your client as well to detect staleness.

For example, here is a way to count stale reads as received by your load test client:

```
# Example tracking structure
last_written_version = {} # key -> version

# After write:
last_written_version[key] = response_version

# After read:
if response_version < last_written_version.get(key, 0):
    stale_read_count += 1
```

You **must** create graphs of your results that show the:

1. Distribution of latency for reads and separately for writes. These graphs should show any "long tail," if it exists.
2. Distribution of the time intervals between reading and writing the same key, as generated by your load-tester.

You must discuss your results. Why do you think the different database configurations gave you those results? Which databases performs best with which kind of read/write ratio? For example, if you had a read-heavy application, which database and which configuration would you use.

Explain how your test generator works. How does your load tester guarantee that reads and writes frequently occur on the same key?

Suggested Deployment Architecture

Deploy all five database nodes as separate EC2 instances.

For Leader-Follower:

- The load tester sends writes directly to the Leader's IP.
- Reads can go to any node, either round-robin implemented in your client, or use an Application Load Balancer (ALB).

For Leaderless:

- Use an ALB in front of all 5 nodes.
- All requests (reads and writes) go through the ALB.

Infrastructure requirements:

- Five EC2 instances for database nodes (t2.micro or t3.micro).
- For Leaderless database: One ALB with target group containing all five nodes.
- Security groups allowing HTTP traffic between nodes and from load tester.
- One EC2 instance for running the load tester. Running the load tester in AWS reduces network effects.

Provide Terraform configurations for the above infrastructure.

Error Handling

For this assignment, assume all nodes are available. If a node is unreachable:

- Log the error.
- Return `503 Service Unavailable` to the client.

You do NOT need to implement retry logic, failure detection, node recovery, or hinted handoff – this is an experimental system, not a production system.

Submission Requirements and Grading

1. **All Code and configurations to be in a Khoury git repository. (40 points total)**
 - a. Put the team; name every team member's name on the report (0.5 points)
 - b. The URL to the code repo. (0.5 points)
 - c. Your repo must be readable by the TAs and Professor
 - d. Java code for the Leader-Follower database

- e. Java code for the Leaderless database
- f. Java code for modified load tester from assignment 2
- g. Correct delays on reads and writes
- h. Correct distribution of reads and writes across nodes in Leaderless database
- i. Dockerfiles
- j. Spring Boot configuration
- k. Terraform configuration
- l. Unit tests that test the three test cases described above
- m. Any other configuration files

2. Explain how your code works (10 points)

- a. Code walk-through videos:
 - i. Explain how the different N/R/W cases are implemented in your code: What happens when a write message arrives? What happens when a read arrives? Don't just describe the code, talk about how the whole thing works, which bits are tricky, how you handle errors etc. **Imagine you are handing this code over to a friend to look after – what would you tell them so that they have an easier time.**
 - ii. These are code walk throughs that will show that you understand what this code does.
 - iii. You **must** have ALL the team members in the walk-through video explaining different aspects of the code.

3. A PDF report with: (40 points total)

For each of the four Leader/Leaderless configurations described in the Load Testing section (10 points each):

- a. Graphs of Time Latencies, ((5+5)*4 points)
 - i. Reads
 - ii. Writes
 - iii. Time interval between reads and writes of the same key.
- b. Discussion of those results (10 points)
 - i. Which type of Leader-Follower performs best with each read/write ratio?
 - ii. Do not just describe the graphs in words, I am looking for reasoned analysis as to why you got those results.
 - iii. Which kind of database would be best used for what kind of application?

As always, the code must be good quality.

Deadline

See Canvas for the deadline.

Remember to turn off all your instances and resources etc in AWS when you finish each day.